# Kinematics Learning

Leandro de Souza Rosa

***Abstract*—...**

***Index Terms*—A, B, C.**

## I. Introduction

## II. Related Works

[1] probably the first proposal of learning forward kinematics with Neural Network (NN). Uses a monolithic network and a planar 2-link robotic arm.

[2] proposes to learn forward kinematics as a monolithic Gaussian Mixture Model (GMM) model. They test on a real robot with (3?)-Degree of Freedom (DoF).

[3] proposed to learn the forward and inverse kinematics together for parallel robots. The approach uses a monolithic model, which is learned online using a Locally Weighted Projection Regression (LWPR).

[4] proposes to solve the forward kinematics using Support Vector Regression (SVR).

[5] proposes to learn a policy with deep-Q learning. They use simulation and robot farms. (fells like I've the images of this paper in another paper. double check)

[6] presents as analysis on sample efficiency for learning the forward kinematics of linked robots. Using tailored parameterized models is efficient, other methods are not sample efficient. They focus on Fourier-Domain representations.

[7] solves the forward kinematics of a 6-DoF serial manipulator using a monolithic NN.

[8] study the influence of joint-position representations on learning forward kinematics with a monolithic NN for a 6-DoF robotic arm and a cylindrical robot. They test `Euler angles`, `vectorial`, `quaternions`, and `angle-axis` representation. Results favor quaternion representation.

[9] proposes learning the inverse kinematics and the dynamics model of serial robot manipulators using Generative Adversarial Networks (GANs).

[10] evaluates different function approximators for learning monolithic forward kinematic models. NNs perform the best.

[11] uses a monolithic NN for learning forward kinematics for a 6-DoF robot manipulator in simulation. The learned model is transferred to a real robot (using the same training signals). Results show that the real robot achieves higher accuracy.

[12] seems to be the same as [11].

[13] uses a monolithic NN for learning forward and inverse kinematics of a 5-DoF robotic arm (check it out it seems they do it for 2 DoFs).

[14] basically lists methods for solving forward kinematics. (does not really add anything)

[15] uses a monolithic NN for learning forward kinematics of a 3-DoF robotic arm.

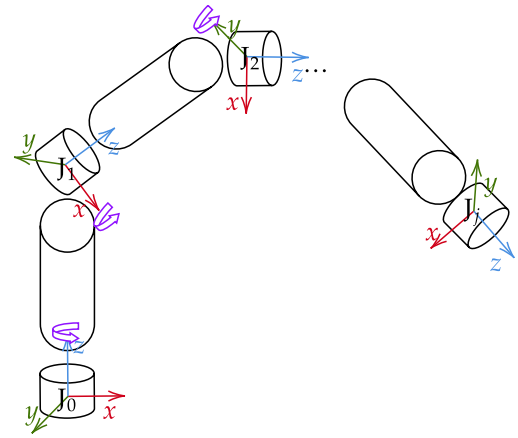


Fig. 1. A serial manipulator with $j$ axes and $j+1$ joints $\mathrm{J}_i$. Each axis is connected by a joint which is associated with a Cartesian reference frame (red, gree, blue) and can rotate over an arbitrary axis (purple). We consider, without loss of generality, the position of $\mathrm{J}_0$ fixed at the world origin, and $\mathrm{J}_j$ to be the end-effector, which does not rotate. The Cartesian position of joint $\mathrm{J}_i | i \in \{1, \ldots, j\}$ ($\mathbf{x}_i$) in the reference frame of $\mathrm{J}_{i_1}$ depends only on $q_{i-1}$.

## III. Preliminaries

Consider the serial robot manipulator illustrated in Figure 1 with $j$ links and $j+1$ joints. The reference frames of the world and of $\mathrm{J}_0$ are fixed and coincide.

Our task is to estimate the Cartesian position of each joint $\mathrm{J}_i | i \in \{1, \ldots, j\}$ in the world reference frame ($\mathbf{x}$) as a function of the rotation angles of $\mathrm{J}_i | i \in \{0, \ldots, j-1\}$ ($q_i | i \in \{0, \ldots, j-1\}$), as described in Equation 1. The Cartesian pose of each joint $\mathrm{J}_i$ is a $d$ dimensional array (usually $n = 2, 3$), resulting in the estimation of $n \times j$ functions in total.

$$\tilde{\mathbf{x}}(\mathbf{q}) = \left[ \begin{array}{cccc} \tilde{\mathbf{x}}_1(q_0) & \tilde{\mathbf{x}}_2(q_0, q_1) & \ldots & \tilde{\mathbf{x}}_j(q_0, \ldots, q_{j-1}) \end{array} \right] \quad (1)$$

The function $\mathbf{x}(\mathbf{q})$ can be expressed using Denavit-Hartenberg (DH) transformation matrices [16]. For a given $\mathrm{J}_i$, the a transformation between its reference frame and the one of $\mathrm{J}_{i-1}$ is described by 2, where $\mathbf{T}$ and $\mathbf{D}$ are the rotation and displacement matrices, respectively.

$$\mathbf{T}^i_{i-1} = \left[ \begin{array}{cc} \mathbf{R}^i_{i-1}(q_{i-1}) & \mathbf{D}^i_{i-1}(q_{i-1}) \\ \mathbf{0}_{1\times 3} & 1 \end{array} \right] \quad (2)$$

For a serial manipulator, the transformations can be chained for obtaining a transformation for any given joint $\mathrm{J}_i$ in the world reference frame, according to Equation 3.

$$\mathbf{x}_i = \prod_{k=1}^{i} \mathbf{T}^k_{k-1} \mid i \in \{1, \ldots, j\} \quad (3)$$

0000–0000/00$00.00 © 2021 IEEE

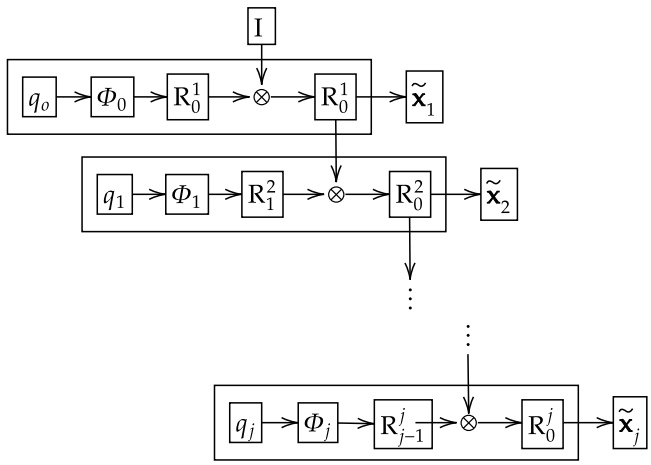


Fig. 2. Proposed architecture. For each joint $i \in \{1, \ldots, j\}$, a network $\Phi_i$ learns the DH transformation from joint $i-1$ to $i$ $\mathbf{R}_{i-1}^i$. $\mathbf{R}_{i-1}^i$ is multiplied by the cumulative transformation of all previous joints, resulting in $\mathbf{R}_0^i$, from which the Cartesian position of joint $i$ $\tilde{\mathbf{x}}_i$ is obtained directly.

## IV. Proposed Architecture

### A. Overview

While approaches in the literature propose to learn a monolithic approximator $\tilde{\mathbf{x}}^m = \Phi^m(\mathbf{q}) : \mathbb{R}^j \rightarrow \mathbb{R}^{n \times j}$, our main idea is to learn one approximator for each joint $\mathrm{J}_i$, which encodes $\mathbf{T}_{i-1}^i$, and thus learning $j$ predictors $\tilde{\mathbf{x}}_i^l = \Phi_i^l(\mathbf{q_i}) : \mathbb{R}^1 \rightarrow \mathbb{R}^{n \times (n+1)} \mid i \in \{1, \ldots j\}$.

### B. Loss Function

For training $\Phi^m$, we follow the approaches in the literature (citation needed)and use the average Euclidean distance between the joints and their respective prediction, as described in Equation 4. Note that some approaches in the literature consider only the end-effector position and its prediction, which is reasonable since in a real robot, the position of individual joints is hard to obtain, while the end-effector's position can be obtained with off-the-shelf trackers and cameras (citation). In contrast, since we perform the training in simulation, we have the flexibility of considering all joints, leading to a richer loss signal, improving the training convergence.

$$\mathcal{L}^m = \frac{1}{j} \sum_{k=1}^{j} ||\tilde{\mathbf{x}}_k^m - \mathbf{x}_k|| \tag{4}$$

Similarly, for each $\Phi_i^l$, we average the Euclidean distance between for all joints up to $\mathrm{J}_i$, according to Equation 5.

$$\mathcal{L}_i^{l,cart} = \frac{1}{i} \sum_{k=1}^{i} ||\tilde{\mathbf{x}}_k^m - \mathbf{x}_k|| \mid i \in \{1, \ldots, j\} \tag{5}$$

Furthermore, we explore two properties of rotation matrices by adding two extra loss terms for $\Phi_i^l$. The first property is that $\det(\mathbf{R}) = 1$, which is captured by Equation 6. The second property is that $\mathbf{R}^T = \mathbf{R}^{-1}$, which is capture by Equation 7.

$$\mathcal{L}_i^{l,det} = ||\det(\tilde{\mathbf{R}}_i) - 1|| \mid i \in \{1, \ldots, j\} \tag{6}$$

$$\mathcal{L}_i^{l,inv} = ||\tilde{\mathbf{R}}_i \tilde{\mathbf{R}}_i^T - \mathbf{I}|| \mid i \in \{1, \ldots, j\} \tag{7}$$

The final loss for training $\Phi_i^l$ is given by Equation 8. The reason for incorporating $\mathcal{L}_i^{l,inv}$ and $\mathcal{L}_i^{l,det}$ in $\mathcal{L}_i^l$ is to motivate $\Phi_i^l$ to converge to valid DH matrices, indirectly pushing $\tilde{\mathbf{D}}_i$ to converge to the proper Cartesian position of $\mathrm{J}_i$.

$$\mathcal{L}_i^l = \mathcal{L}_i^{l,cart} + \mathcal{L}_i^{l,inv} + \mathcal{L}_i^{l,det} \tag{8}$$

(add the division by the sum of times lq is computed)
(probably should prepare an ablation figure for these losses)

### C. Training Procedure

Since in this stage we use simulation, both $\Phi^m$ and $\Phi^l$ are trained in a straight-forward fashion according to Algorithm 1. For each epoch of training, $nSamples$ random poses are sampled in joint space (line 3), and the respective Cartesian position of each joint is obtained directly from the simulator (line 4) and added to a memory buffer (line 5). The memory size is limited, and upon additions further than its limit, it drops the oldest data. Next, $nIterations$ of training is done with the data already within the memory, which are done by randomly sampling $batchSize$ samples from the memory (line 8), computing the average loss (lines 9 to 13), which is then back-propagated through the network (line 14).

**Algorithm 1:** Training procedure for $\Phi$.

1 **for** $e \in \{1, \ldots, epochs\}$ **do**
2  **for** $r \in \{1, \ldots, nSamples\}$ **do**
3   $q \leftarrow$ random joint positions;
4   $x \leftarrow$ Cartesian position of all joints given $q$;
5   add $(q, x)$ to $memory$;
6  **end**
7  **for** $t \in \{1, \ldots, nIterations\}$ **do**
8   $(qs, xs) \leftarrow$ randomly select $batchSize$ samples from $memory$;
9   $meanLoss \leftarrow 0$;
10   **for** $(q, x) \in (qs, xs)$ **do**
11    $meanLoss \leftarrow acc + \mathcal{L}$;
12   **end**
13   $meanLoss \leftarrow acc/batchSize$;
14   backpropagate $meanLoss$ through $\Phi$;
15  **end**
16 **end**

## V. Results

## Acknowledgements

This work was supported by ....

## References

[1] L. Nguyen, R. Patel, and K. Khorasani, "Neural network architectures for the forward kinematics problem in robotics," in *1990 IJCNN International Joint Conference on Neural Networks*, 1990, pp. 393–399 vol.3.

[2] E. Gribovskaya, S. Khansari-Zadeh, and A. Billard, "Learning nonlinear multivariate dynamics of motion in robotic manipulators," *The International Journal of Robotics Research*, vol. 30, no. 1, pp. 80–117, 2011. [Online]. Available: https://doi.org/10.1177/0278364910376251

[3] B. Damas and J. Santos-Victor, "An online algorithm for simultaneously learning forward and inverse kinematics," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 1499–1506.

[4] A. Morell, M. Tarokh, and L. Acosta, "Solving the forward kinematics problem in parallel robots using support vector regression," *Engineering Applications of Artificial Intelligence*, vol. 26, no. 7, pp. 1698–1706, 2013. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0952197613000535

[5] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 3389–3396.

[6] D. Kubus, R. Rayyes, and J. J. Steil, "Learning forward and inverse kinematics maps efficiently," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 5133–5140.

[7] M. Theofanidis, S. I. Sayed, J. Cloud, J. Brady, and F. Makedon, "Kinematic estimation with neural networks for robotic manipulators," in *Artificial Neural Networks and Machine Learning – ICANN 2018*, V. Kůrková, Y. Manolopoulos, B. Hammer, L. Iliadis, and I. Maglogiannis, Eds. Cham: Springer International Publishing, 2018, pp. 795–802.

[8] R. M. Grassmann and J. Burgner-Kahrs, "On the merits of joint space and orientation representations in learning the forward kinematics in se(3)," in *Proceedings of Robotics: Science and Systems*, Freiburgim-Breisgau, Germany, June 2019.

[9] H. Ren and P. Ben-Tzvi, "Learning inverse kinematics and dynamics of a robotic manipulator using generative adversarial networks," *Robotics and Autonomous Systems*, vol. 124, p. 103386, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0921889019303501

[10] I. M. Loutfi, A. B. Boutchouang, A. Melingui, O. Lakhal, F. B. Motto, and R. Merzouki, "Learning-based approaches for forward kinematic modeling of continuum manipulators," *IFAC-PapersOnLine*, vol. 53, no. 2, pp. 9899–9904, 2020, 21st IFAC World Congress. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2405896320334595

[11] K. P. and B. B. Choudhury, "Transfer Learning-Based Artificial Neural Network for Forward Kinematic Estimation of 6-DOF Robot," *International Journal of Applied Metaheuristic Computing (IJAMC)*, vol. 13, no. 1, pp. 1–15, January 2022. [Online]. Available: https://ideas.repec.org/a/igg/jamc00/v13y2022i1p1-15.html

[12] P. Kesaba and B. Choudhury, "Forward kinematic analysis of 6-r industrial robot using active sampled neural network regressor," *International Journal of Software Innovation*, vol. 10, no. 1, pp. 1–14, 2022 2022. [Online]. Available: https://www.proquest.com/scholarly-journals/forward-kinematic-analysis-6-r-industrial-robot/docview/2904627281/se-2

[13] A.-N. Sharkawy, "Forward and inverse kinematics solution of a robotic manipulator using a multilayer feedforward neural network," *Journal of Mechanical and Energy Engineering*, vol. 6, no. 2, Oct. 2022. [Online]. Available: https://jmee.tu.koszalin.pl/ojs/index.php/jmee/article/view/300

[14] T. P. Valayil and R. S. Augustine, "Methods to solve forward kinematics of parallel and serial manipulators," in *AIP Conference Proceedings*, vol. 2670, no. 1. AIP Publishing, 2022.

[15] A.-N. Sharkawy and S. Khairullah, "Forward and inverse kinematics solution of a 3-dof articulated robotic manipulator using artificial neural network," *International Journal of Robotics and Control Systems*, vol. 3, no. 2, pp. 330–353, 2023. [Online]. Available: https://www.pubs2.ascee.org/index.php/IJRCS/article/view/1017

[16] S. Kucuk and Z. Bingul, *Robot Kinematics: Forward and Inverse Kinematics*, S. Cubero, Ed. InTech, 2006.